

Prediksi Penjualan Dan Poupularitas Game

Makalah ini disusun untuk memenuhi tugas mata kuliah
Kecerdasan Artificial



Disusun oleh:

Muh. Argya Ishmat Islah Putra Mansyur	24031554042
Yuli Inka Valentina Simanjuntak	24031554118
Nagatan Alief Putra Silahen	24031554086

Dosen Pengampu:

Riskyana Dewi Intan Puspitasari, M.Kom.
Yuni Rosita Dewi, M.Si.

**PROGRAM STUDI S1 SAINS DATA
FAKULTAS MATEMATIKA & ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
2025**

BAB I

PENDAHULUAN

A. Latar Belakang

Dalam beberapa tahun terakhir, industri *game* berkembang dengan sangat pesat. Persaingan yang semakin ketat menuntut setiap pengembang (*developer*) dan penerbit untuk menciptakan *game* yang tidak hanya berkualitas tinggi, tetapi juga sesuai dengan selera pemain saat ini. Kesuksesan sebuah *game* kini tidak bisa hanya dilihat dari satu sisi saja, melainkan dipengaruhi oleh banyak faktor, mulai dari genre, harga, ulasan pengguna, hingga spesifikasi teknisnya. Hal inilah yang membuat prediksi mengenai popularitas atau angka penjualan sebuah *game* menjadi tantangan tersendiri.

Keragaman dan kompleksitas faktor tersebut membuat proses menilai potensi kesuksesan sebuah *game*, baik dari sisi popularitas maupun penjualan yang menjadi sebuah tantangan yang signifikan. *Developer* dan *publisher* sering kali menghadapi risiko tinggi ketika merilis *game* baru, terutama ketika keputusan dilakukan tanpa dukungan data yang kuat. Kesalahan dalam menentukan harga, genre yang kurang diminati, hingga penentuan platform yang tidak sesuai dapat berdampak pada rendahnya performa *game* di pasar.

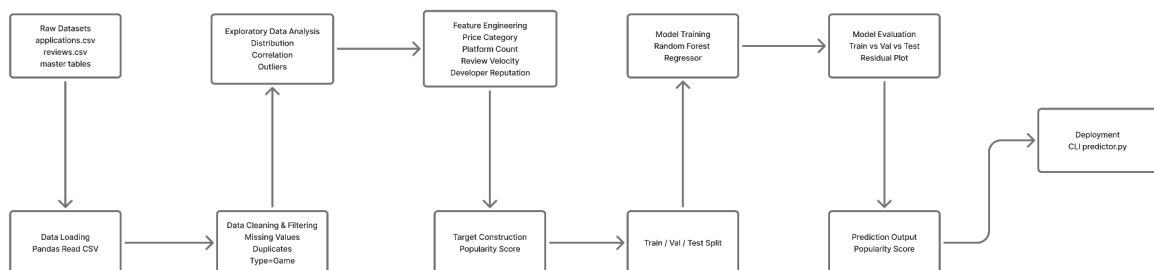
Seiring dengan meningkatnya ketersediaan data dan kemajuan teknologi kecerdasan buatan, pendekatan analisis berbasis data mulai menjadi solusi yang sangat relevan dan efektif. Melalui algoritma *Machine Learning* seperti *Random Forest Regressor*, *Gradient Boosting* (XGBoost), dan *Neural Network* (MLP), pola-pola tersembunyi dalam data *game* yang telah dirilis sebelumnya dapat diidentifikasi dan dianalisis. Model-model tersebut mampu mempelajari hubungan kompleks antar-fitur, sehingga menghasilkan prediksi yang lebih akurat terkait potensi popularitas maupun estimasi penjualan sebuah *game* baru.

Dengan memanfaatkan pendekatan ini, pengembang dapat memiliki gambaran yang lebih jelas mengenai potensi pasar sebelum melakukan perilisan *game*. Prediksi berbasis AI tidak hanya membantu mengurangi risiko kerugian, tetapi juga mendukung pengambilan keputusan strategis, seperti menentukan strategi pemasaran, menetapkan harga yang kompetitif, memilih platform rilis yang tepat, serta merancang fitur yang lebih sesuai dengan kebutuhan pemain. Oleh karena itu, analisis prediktif menggunakan *Machine Learning* menjadi semakin penting dalam industri *game* modern, terutama dalam upaya menghasilkan *game* yang relevan, kompetitif, dan memiliki peluang tinggi untuk sukses di pasar.

B. Ide Project Secara Umum

Berdasarkan latar belakang di atas, proyek ini mengusulkan pengembangan sistem prediktif berbasis *Artificial Intelligence* (AI) untuk mengestimasi kesuksesan sebuah *game* baru. Secara garis besar, ide proyek ini dapat dijabarkan sebagai berikut:

1. Sumber Data (*Dataset*): Penelitian ini akan menggunakan *dataset* terbaru yang bersumber dari platform *Steam* (tahun 2025). Data ini mencakup ribuan judul *game* dengan berbagai atribut yang relevan dengan kondisi pasar terkini.
2. Fitur Prediksi: Model AI akan dilatih menggunakan kombinasi fitur numerik dan kategorikal untuk menilai pola kesuksesan. Variabel independen (fitur) yang digunakan meliputi:
 - Harga: Menilai pengaruh harga jual terhadap volume pemain.
 - *Rating & Jumlah Review*: Indikator penerimaan komunitas terhadap kualitas *game*.
 - *Developer & Publisher*: Mengukur pengaruh reputasi pengembang.
 - Genre & Tags: Mengkategorikan jenis permainan yang sedang tren.
 - Platform: Ketersediaan sistem operasi (Windows, Mac, Linux).
3. Metode & Algoritma: Untuk mendapatkan hasil prediksi yang optimal, proyek ini akan mengkomparasi dan menerapkan algoritma regresi yang handal, antara lain:
 - *Random Forest Regressor*: Untuk menangani data non-linear dengan variabilitas tinggi.
 - *Gradient Boosting (XGBoost)*: Untuk meningkatkan akurasi prediksi melalui metode *ensemble*.
 - *Neural Network (MLP)*: Sebagai pendekatan *deep learning* pada data tabular untuk menangkap pola kompleks.
4. Luaran (*Output*): Hasil akhir dari model ini adalah skor popularitas atau estimasi jumlah pemain. *Output* ini diharapkan dapat memberikan wawasan bisnis (*business insight*) yang nyata bagi pengembang dalam memproyeksikan performa *game* mereka sebelum diluncurkan ke pasar.



C. Ide Project Secara Umum

1. Membangun model AI yang dapat memprediksi tingkat popularitas dan potensi penjualan game berdasarkan fitur seperti genre, platform, harga, developer, dan rating.
2. Menganalisis faktor-faktor utama yang paling berpengaruh terhadap kesuksesan game menggunakan algoritma seperti *Random Forest*, *XGBoost*, dan *MLP*.
3. Menghasilkan skor prediksi berupa estimasi jumlah pemain, penjualan, dan tingkat popularitas sebagai output evaluasi game sebelum dirilis.

4. Menyediakan sistem pendukung keputusan yang dapat membantu *developer* dan *publisher* menilai potensi pasar dan mengoptimalkan strategi produksi serta pemasaran.

D. Manfaat

1. Mengurangi risiko kegagalan game dengan memberikan gambaran potensi pasar sejak tahap perencanaan.
2. Membantu *developer/publisher* menentukan strategi harga, genre yang sesuai tren, dan target pasar berdasarkan prediksi popularitas.
3. Memberikan *insight* bisnis mengenai elemen apa yang paling mempengaruhi keberhasilan game sehingga perusahaan dapat fokus pada fitur yang benar-benar penting.
4. Mendukung penelitian dan edukasi sebagai contoh aplikasi nyata machine learning dalam industri gaming dan analisis data berbasis AI.

BAB II

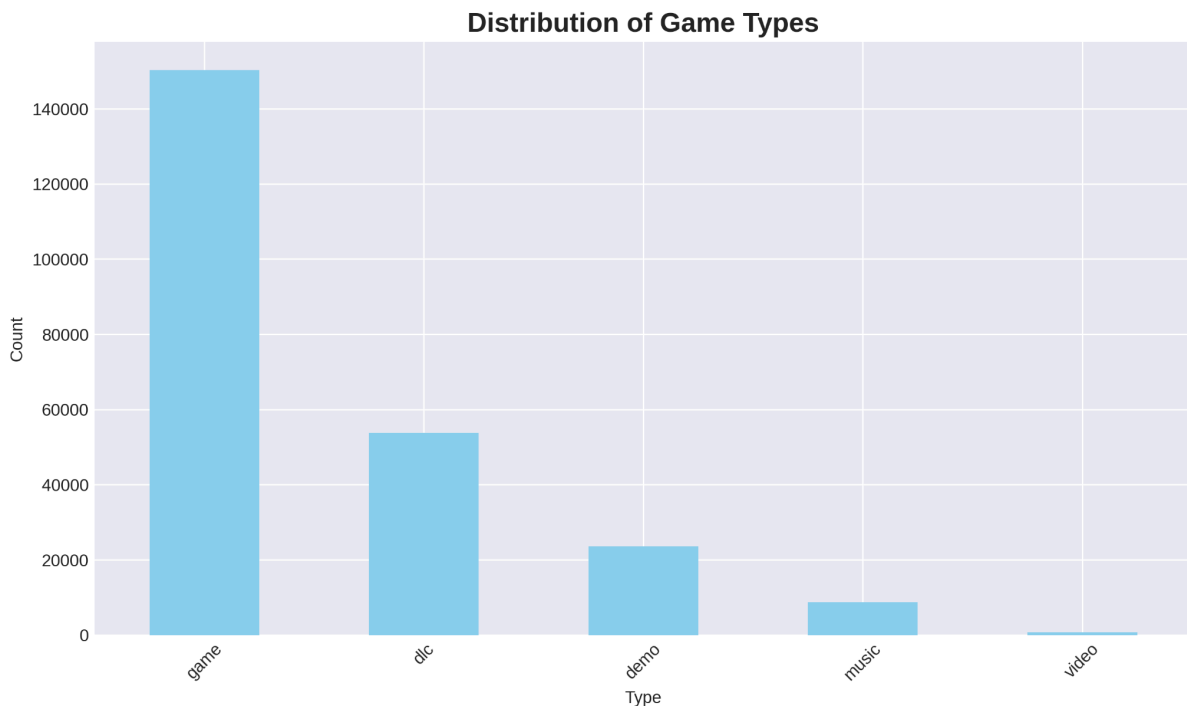
EXPLORATORY DATA ANALYSIS (EDA) & PRA - PEMROSESAN DATA

A. Analisis Eksploratif (EDA)

1. Distribusi Variabel Kunci

1.1 Distribusi Tipe Aplikasi

Tahap awal eksplorasi data dilakukan dengan memetakan variasi produk yang tersedia di platform Steam. Mengingat Steam merupakan ekosistem yang luas, dataset mentah yang ditemukan tidak hanya berisi permainan (*game*), melainkan juga mencakup konten tambahan (*DLC*), *soundtrack*, hingga perangkat lunak pendukung (*tools*).



Gambar 1.1 Tipe distribusi game

Visualisasi pada grafik Gambar 1.1 menunjukkan bahwa kategori 'game' memang mendominasi dataset, yang mana hal ini sangat wajar mengingat profil utama Steam. Namun, keberadaan entitas non-game tetap perlu diperhatikan karena karakteristik pasar mereka sangat berbeda. Sebagai contoh, volume ulasan pada sebuah *soundtrack* biasanya lebih dipengaruhi oleh popularitas game induknya daripada kualitas musik itu sendiri secara independen.

Oleh karena itu, saya mengambil langkah teknis untuk memfilter data dan hanya menyertakan entitas bertipe 'game' pada analisis berikutnya. Keputusan ini krusial untuk menjaga akurasi model karena dua alasan utama. Pertama, untuk meminimalisir *noise*; menyertakan *DLC* atau *tools* dalam pemodelan hanya akan mengaburkan pola prediksi, karena fitur seperti *achievements* atau *playtime*

pada item tersebut seringkali tidak tersedia atau tidak relevan. Kedua, untuk menjamin konsistensi data, sehingga perbandingan yang dilakukan bersifat *apple-to-apple*. Dengan dataset yang homogen, model AI diharapkan mampu mengenali variabel yang benar-benar berpengaruh terhadap kesuksesan sebuah judul game secara lebih akurat.

1.2 Harga vs Paid

Tahapan eksplorasi data selanjutnya difokuskan pada pengelompokan judul game berdasarkan model bisnisnya, yaitu game berbayar (*paid*) dan game gratis (*free*).



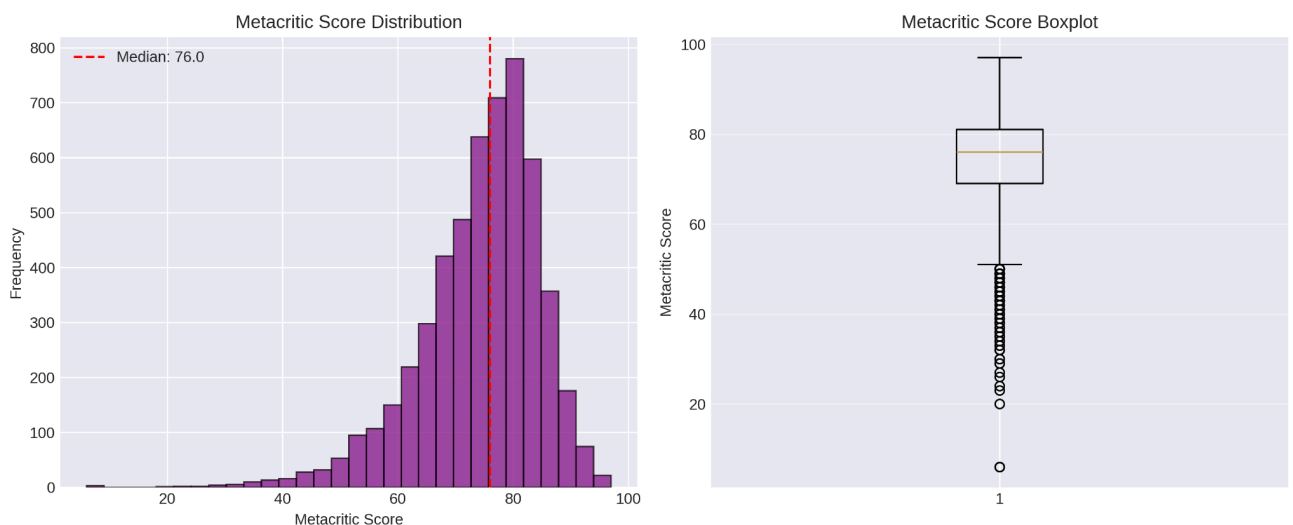
Gambar 1.2 Distribusi Free vs Paid

Melalui visualisasi *pie chart* pada grafik gambar 1.2, terlihat pola distribusi pasar yang cukup kontras di mana mayoritas game di Steam, yakni sekitar 80%, merupakan game berbayar. Meskipun game berbayar mendominasi secara kuantitas dalam dataset, kelompok game gratis yang hanya berjumlah sekitar 20% justru sering kali mencatatkan angka popularitas yang jauh lebih fenomenal. Hal ini terjadi karena game gratis tidak memiliki hambatan biaya masuk (*price barrier*), sehingga memungkinkan penetrasi pasar yang lebih cepat dan akumulasi jumlah rekomendasi yang masif dari basis pemain yang sangat luas.

Pemisahan kategori ini memiliki signifikansi teknis yang besar dalam pembangunan model AI. Variabel *is_free* bertindak sebagai fitur kunci yang membantu algoritma memahami potensi keterlibatan (*engagement*) pengguna sejak awal. Model akan mempelajari pola bahwa status "gratis" cenderung menjadi pendorong utama bagi viralitas sebuah game. Dengan memasukkan data ini, model AI tidak hanya sekadar melihat harga sebagai angka, tetapi juga sebagai strategi aksesibilitas yang memengaruhi perilaku pemain dalam memberikan ulasan dan rekomendasi. Integrasi fitur ini sangat krusial agar prediksi keberhasilan game yang dihasilkan nantinya menjadi lebih akurat dan relevan dengan dinamika ekonomi di platform Steam.

1.3 Analisis Kualitas Melalui Skor Metacritic

Selain ulasan dari pengguna, kualitas sebuah permainan di platform Steam juga direpresentasikan melalui skor Metacritic, yang merupakan agregasi penilaian dari para kritikus profesional. Berdasarkan analisis pada dataset, ditemukan fakta menarik bahwa hanya sekitar 2,2% atau sebanyak 5.299 judul game yang memiliki skor Metacritic. Rendahnya persentase ini menunjukkan bahwa penilaian profesional adalah aset yang langka dan biasanya hanya tersedia untuk game yang memiliki tingkat eksposur atau popularitas tertentu di pasar. Meskipun cakupannya kecil, variabel ini menjadi salah satu metrik kualitas paling prestisius yang dapat memengaruhi persepsi calon pembeli secara signifikan.



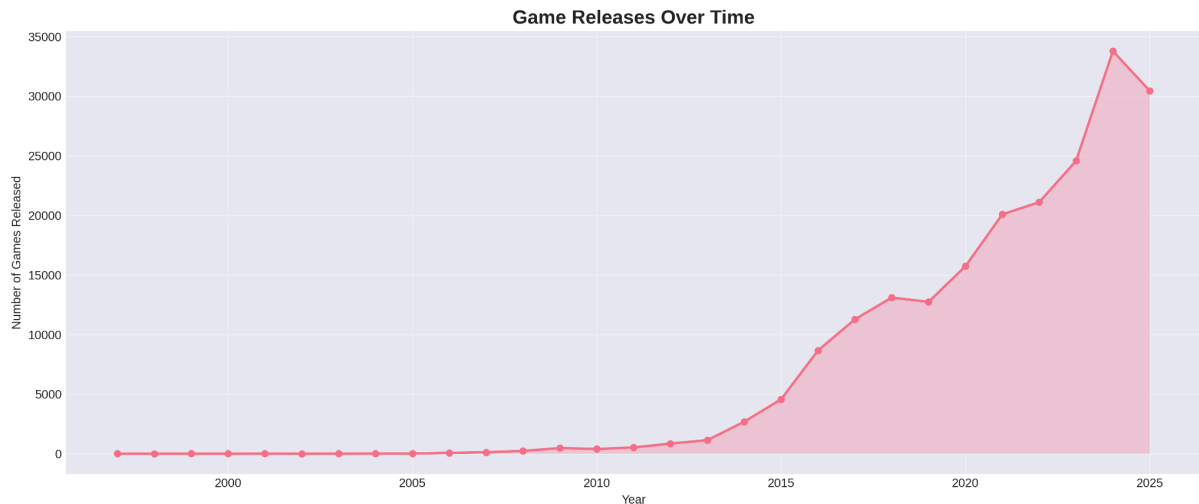
Gambar 1.3 Distribusi Skor Metacritic

Secara statistik, distribusi skor tersebut menunjukkan nilai rata-rata di angka 74, dengan mayoritas data terkonsentrasi secara padat pada rentang nilai 70 hingga 85. Visualisasi pada grafik gambar 1.3 memperlihatkan adanya beberapa *outliers* atau pencilan pada kedua ujung skala. Keberadaan game dengan skor sangat rendah (di bawah 40) atau skor sangat tinggi (di atas 90) memberikan informasi krusial bagi model AI untuk mengenali anomali kualitas. Skor di atas 90, misalnya, sering kali menjadi sinyal kuat bagi sebuah game "blockbuster" yang hampir selalu berkorelasi langsung dengan lonjakan jumlah rekomendasi dan basis pemain yang masif. Dengan mengintegrasikan data ini, model dapat membedakan antara game berkualitas standar dengan game berkualitas tinggi yang memiliki potensi sukses besar di pasar global.

2. Tren Temporal dan Hubungan Antar Fitur

2.1 Analisis Tren Rilis Game (1990 - 2025)

Analisis temporal dilakukan untuk melacak volume publikasi judul game di platform Steam selama rentang waktu lebih dari tiga dekade.



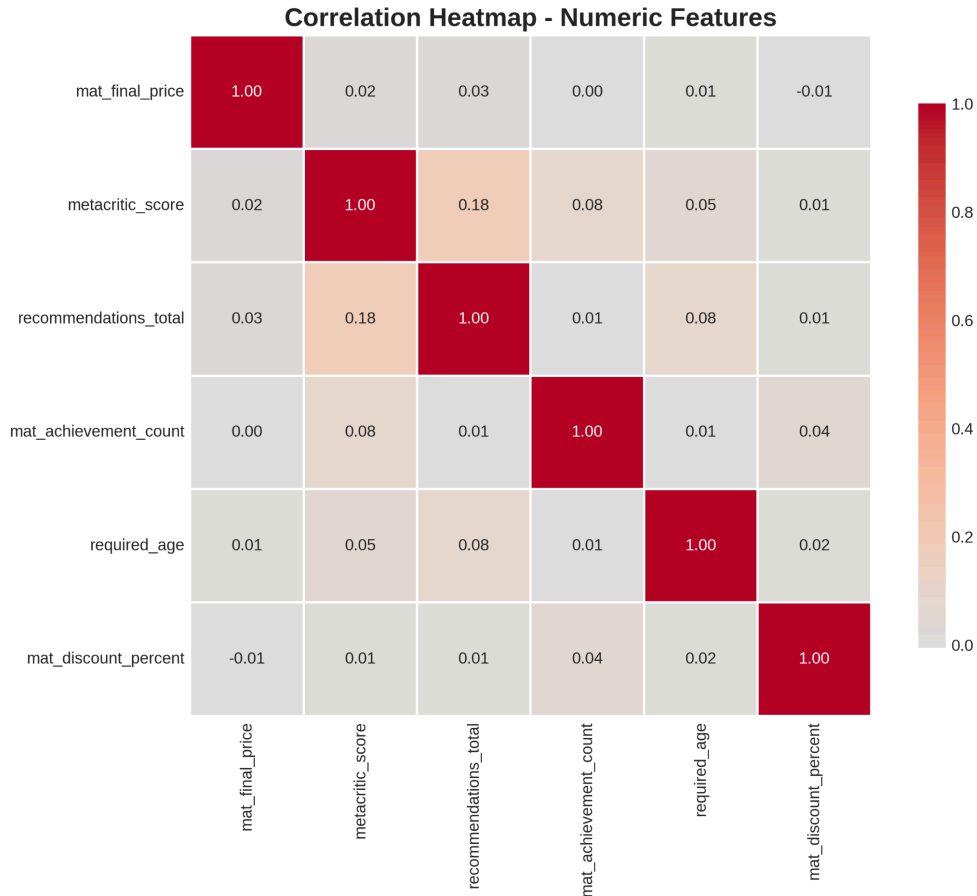
Gambar 2.1 Rilis Game Waktu ke Waktu

Melalui grafik garis gambar 2.1, kita dapat mengamati perjalanan industri game dari masa ke masa. Data menunjukkan bahwa pada periode 1990-an hingga awal 2010-an, jumlah rilis game cenderung stabil namun tumbuh secara perlahan. Namun, terjadi titik balik yang sangat signifikan mulai sekitar tahun 2014, di mana grafik menunjukkan lonjakan pertumbuhan yang bersifat eksponensial. Fenomena ini mencerminkan demokratisasi dalam industri pengembangan game, yang dipicu oleh kemudahan akses *self-publishing* (seperti program Steam Greenlight dan kemudian Steam Direct) serta maraknya penggunaan *game engine* yang lebih terjangkau oleh pengembang indie.

Lonjakan jumlah rilis ini mengindikasikan bahwa pasar telah mencapai tingkat kejenuhan (*saturated*) yang tinggi, di mana ribuan pengembang kini bersaing ketat untuk memperebutkan perhatian audiens yang sama. Dalam konteks pembangunan model AI, variabel tahun rilis ini memegang peranan yang sangat vital sebagai faktor normalisasi. Terdapat bias akumulasi waktu yang sangat nyata dalam dataset; sebuah game yang dirilis pada tahun 2010 secara teknis memiliki waktu lebih dari satu dekade untuk mengumpulkan ulasan dan rekomendasi, dibandingkan dengan game baru yang dirilis pada tahun 2023. Jika variabel waktu ini diabaikan, model akan cenderung memberikan penilaian yang tidak adil hanya berdasarkan volume angka mentah. Oleh karena itu, data tren rilis ini digunakan oleh model untuk menyeimbangkan bobot prediksi, memastikan bahwa potensi kesuksesan sebuah game diukur secara proporsional berdasarkan durasi paparan (*exposure time*) game tersebut di pasar.

2.2 Analisis Korelasi Antar Variabel Numerik

Langkah penutup dalam analisis eksploratif ini adalah membedah hubungan antar variabel numerik menggunakan matriks korelasi yang divisualisasikan lewat *heatmap*.



Gambar 2.2 Korelasi - Fitur Numerik

Melalui grafik gambar 2.2, kita bisa memetakan sejauh mana variabel seperti harga, skor Metacritic, dan jumlah *achievement* saling berkaitan dengan total rekomendasi sebagai tolok ukur kesuksesan. Dari sini, akan terlihat jelas mana variabel yang memiliki pengaruh nyata dan mana yang sebenarnya berdiri sendiri tanpa memengaruhi variabel lainnya.

Ada beberapa temuan menarik yang didapatkan dari analisis ini. Pertama, skor Metacritic terhadap jumlah rekomendasi memiliki korelasi positif di angka sekitar 0.18. Meski berhubungan, pengaruhnya ternyata tidak terlalu dominan, yang berarti nilai bagus dari kritikus memang membantu meningkatkan popularitas tetapi bukan satu-satunya penentu; faktor lain seperti genre atau momentum pemasaran bisa jadi jauh lebih berpengaruh dalam menciptakan viralitas. Kedua, ditemukan semacam "paradoks harga" di mana korelasi antara harga game dan jumlah rekomendasi tergolong sangat rendah. Hal ini mematahkan anggapan bahwa game mahal (seperti kelas AAA) pasti akan selalu mendapatkan lebih banyak rekomendasi. Sebaliknya, data justru memperkuat temuan sebelumnya bahwa banyak game dengan harga ekonomis yang mampu meraih tingkat *engagement* jauh lebih masif.

Secara teknis, hasil dari *heatmap* ini menjadi fondasi penting dalam tahap pemilihan fitur (*feature selection*) untuk menghindari masalah *multikolinearitas*. Jika terdapat dua fitur yang memiliki korelasi sangat tinggi di atas 0.9 seperti total ulasan dan jumlah rekomendasi, maka salah satunya harus dieliminasi. Langkah ini diambil agar model AI tidak mendapatkan informasi yang tumpang tindih (redundansi) yang bisa memicu *overfitting*. Dengan begitu, model yang dibangun nantinya akan jauh

lebih stabil, efisien, dan benar-benar hanya memproses variabel yang memberikan kontribusi informasi unik bagi prediksi kesuksesan sebuah game.

3. Analisis Outlier dan Identifikasi Fitur Dominan

Langkah terakhir dalam tahap eksplorasi ini adalah mendeteksi keberadaan pencilan (*outlier*) serta melakukan pemetaan awal terhadap variabel yang paling berpengaruh bagi model.

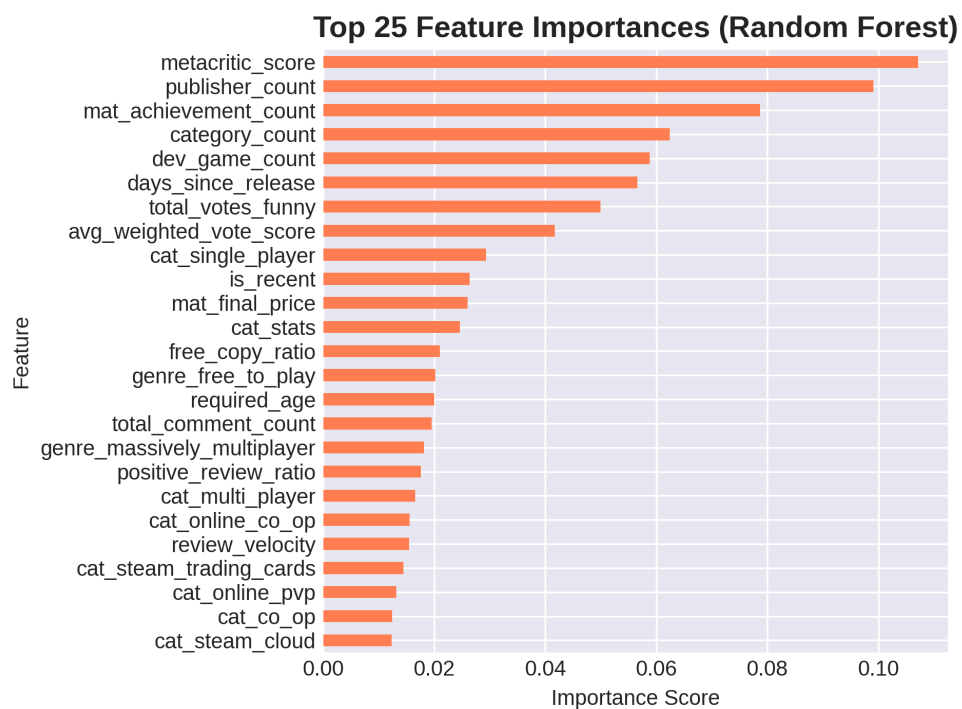
3.1. Deteksi "Game Superstar" melalui Analisis Outlier

Dalam industri game di Steam, distribusi popularitas biasanya tidak merata atau sangat timpang. Melalui visualisasi *boxplot*, ditemukan banyak sekali pencilan pada variabel jumlah rekomendasi. Sebagian besar game di Steam hanya memiliki sedikit interaksi, namun ada segelintir game "superstar" yang memiliki jutaan rekomendasi.

Alih-alih menganggap pencilan ini sebagai gangguan (*noise*), data ini justru memberikan gambaran nyata tentang profil game yang berhasil mencetak sukses besar. Dengan mengidentifikasi batas-batas ekstrem ini, saya bisa memahami karakteristik unik yang membedakan game populer dengan game rata-rata. Pemahaman tentang *outlier* ini sangat membantu dalam menyiapkan model agar tidak hanya akurat dalam memprediksi game skala kecil, tetapi juga mampu mengenali pola pertumbuhan game-game yang berpotensi viral.

3.2 Peringkat Kepentingan Fitur (Initial Feature Importance)

Sebelum masuk ke tahap permodelan yang lebih kompleks, saya menggunakan algoritma *Random Forest* untuk melakukan pengecekan awal mengenai fitur mana yang paling dominan dalam menentukan target prediksi. Langkah ini bertujuan untuk memvalidasi apakah fitur-fitur yang telah disiapkan memang memiliki "tenaga" dalam memberikan prediksi.



Gambar 3.2 Top 25 Fitur Important

Dari grafik gambar 3.2, terlihat jelas variabel mana yang memberikan kontribusi paling besar. Hasilnya seringkali menunjukkan bahwa fitur seperti jumlah ulasan positif dan skor Metacritic berada di posisi teratas. Temuan ini sangat krusial karena mengonfirmasi bahwa kualitas produk dan reputasi di mata pengguna adalah pendorong utama kesuksesan. Dengan mengetahui peringkat kepentingan ini, saya bisa lebih fokus dalam menyempurnakan fitur-fitur kunci tersebut dan menyederhanakan variabel lain yang kurang relevan, sehingga kinerja model AI nantinya menjadi lebih tajam dan efisien.

B. Data Cleaning

Setelah memahami karakteristik data melalui tahap eksplorasi, langkah selanjutnya yang sangat krusial adalah pembersihan data. Proses ini dilakukan untuk mengeliminasi *noise* dan ketidakkonsistenan yang dapat mengganggu performa model AI. Langkah pertama dalam tahap ini adalah melakukan penyaringan ketat terhadap jenis entitas. Berdasarkan temuan pada tahap EDA, dataset hanya difokuskan pada entitas bertipe 'game'. Keputusan ini diambil untuk memastikan model hanya mempelajari pola dari produk permainan mandiri, sementara entitas lain seperti *software*, *tools*, atau *soundtrack* (musik) dibuang karena memiliki struktur data dan perilaku pasar yang sangat berbeda.

Masalah data yang hilang (*missing values*) ditangani dengan pendekatan yang spesifik dan konservatif guna menjaga integritas informasi. Untuk variabel numerik yang berkaitan dengan performa dan teknis, seperti *metacritic_score*, *recommendations_total*, dan *mat_achievement_count*, nilai yang kosong diisi dengan 0. Logika di balik pengisian ini adalah untuk menghindari estimasi yang berlebihan; jika suatu game tidak memiliki catatan skor atau rekomendasi, maka dianggap game tersebut belum mencapai ambang batas minimum untuk tercatat. Hal serupa diterapkan pada variabel ekonomi seperti harga (*mat_final_price*) dan persentase diskon, di mana nilai kosong diasumsikan sebagai angka nol. Sementara itu, untuk kolom dukungan platform (Windows, Mac, Linux), nilai *null* secara otomatis dikonversi menjadi False untuk menegaskan ketiadaan dukungan pada sistem operasi tersebut.

Selain data numerik, standarisasi pada data teks juga menjadi perhatian utama untuk menjaga konsistensi input bagi model. Setiap informasi dalam kolom nama dan deskripsi game diproses melalui transformasi huruf kecil (lowercase) dan penghapusan spasi berlebih (*strip*). Hal ini dilakukan agar sistem tidak menganggap dua teks yang identik sebagai entitas yang berbeda hanya karena perbedaan penggunaan huruf kapital atau spasi. Untuk kolom teks atau kategori yang benar-benar kosong, pengisian dilakukan dengan menggunakan *empty string* atau label "Unknown". Seluruh rangkaian pembersihan ini memastikan bahwa dataset yang masuk ke tahap pemodelan adalah data yang bersih, seragam, dan representatif terhadap kondisi pasar yang sebenarnya.

C. Integrasi & Rekayasa Fitur (Feature Engineering)

Tahap rekayasa fitur merupakan langkah paling menentukan untuk meningkatkan daya prediksi model AI. Pada tahap ini, data mentah diubah menjadi informasi yang lebih bermakna melalui penggabungan dataset dan pembuatan variabel baru yang merepresentasikan aspek kualitas serta popularitas secara lebih akurat.

A. Agregasi Metrik Performa dari Data Ulasan Langkah pertama melibatkan integrasi antara dataset aplikasi utama dengan data ulasan pengguna yang bersumber dari `reviews.csv`. Proses penggabungan ini menghasilkan tiga metrik performa baru yang memberikan gambaran mendalam tentang interaksi pengguna. *Positive Review Ratio* dihitung sebagai perbandingan antara ulasan positif terhadap total ulasan untuk menangkap tingkat kepuasan pemain secara objektif. Selanjutnya, dilakukan konversi *Total Playtime* dari satuan menit ke jam guna mempermudah interpretasi data oleh model terkait kedalaman konten game. Terakhir, dibentuk variabel *Engagement Score* yang merangkum intensitas interaksi pengguna berdasarkan akumulasi suara dukungan (*upvotes*) dan jumlah komentar pada ulasan. Gabungan metrik ini memungkinkan model untuk tidak hanya melihat jumlah pemain, tetapi juga kualitas hubungan antara game dan komunitasnya.

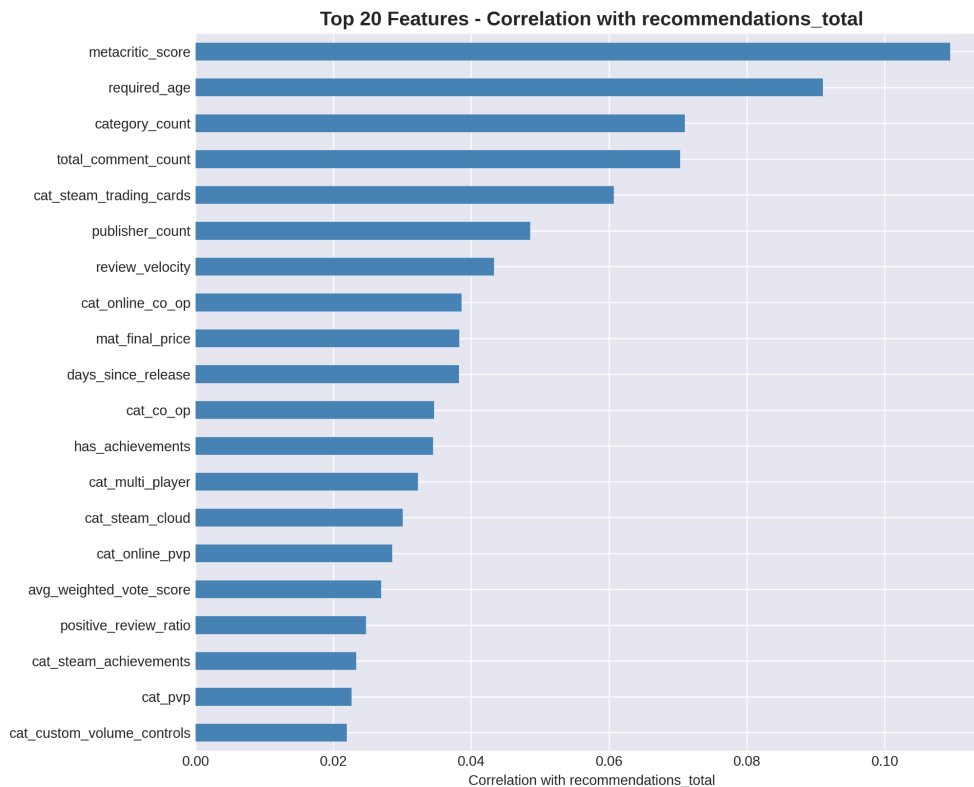
B. Pembuatan Fitur Reputasi dan Skala Pengembang Industri game sangat dipengaruhi oleh rekam jejak pembuatnya, sehingga dibuatlah fitur reputasi untuk menangkap konteks historis tersebut. Fitur *Developer Reputation* (`dev_avg_popularity`) dihitung berdasarkan rata-rata jumlah rekomendasi dari judul-judul game yang pernah dirilis sebelumnya oleh pengembang yang sama. Untuk menjaga validitas, fitur ini hanya diberikan kepada pengembang yang telah merilis minimal dua game, guna menghindari bias pada pengembang baru. Selain reputasi, dilakukan kategorisasi *Publisher Size* yang membagi pengembang ke dalam empat skala: *Indie* (1 game), *Small* (1-5), *Medium* (5-10), dan *Large* (lebih dari 10 game). Penambahan fitur ini sangat krusial agar model dapat membedakan pola kesuksesan antara studio independen kecil dengan perusahaan penerbit besar yang memiliki sumber daya pemasaran lebih luas.

C. Transformasi Teknis: Encoding dan Normalisasi Agar seluruh data dapat diproses secara matematis oleh algoritma *Neural Network* dan *XGBoost*, dilakukan transformasi teknis pada variabel kategorikal dan numerik. Mengingat banyaknya variasi genre dan kategori di Steam, diterapkan *One-Hot Encoding* khusus pada *Top 20 Genre* dan *Top 20 Categories* yang paling umum muncul. Strategi ini diambil untuk mencegah masalah *dimensionality explosion* (jumlah kolom yang terlalu banyak) namun tetap mempertahankan informasi yang paling relevan. Sebagai langkah final, seluruh fitur numerik diproses menggunakan *StandardScaler*. Proses ini menormalisasi data sehingga memiliki rata-rata (*mean*) nol dan standar deviasi satu. Normalisasi ini sangat penting untuk memastikan bahwa fitur dengan rentang nilai besar (seperti jumlah ulasan) tidak mendominasi fitur dengan rentang kecil (seperti harga) dalam perhitungan model, sehingga proses pelatihan menjadi lebih stabil dan konvergen.

D. Seleksi Fitur

Tahap seleksi fitur merupakan proses krusial yang bertujuan untuk menyaring variabel paling relevan, guna meningkatkan efisiensi komputasi model sekaligus mempertajam akurasi prediksi. Langkah pertama dimulai dengan melakukan Analisis Multikolinearitas yang ketat. Dalam proses ini, dilakukan pemeriksaan terhadap pasangan fitur independen untuk mendeteksi adanya hubungan yang terlalu kuat (*redundansi*). Fitur yang memiliki nilai korelasi di atas 0.9 dieliminasi salah satunya, karena dua variabel yang tumpang tindih informasinya hanya akan menambah beban model tanpa memberikan nilai tambah pada prediksi. Hal ini penting untuk menjaga stabilitas model dan mencegah terjadinya *overfitting*, sehingga model AI tetap ramping namun berdaya guna tinggi.

Selanjutnya, dilakukan pemetaan melalui Korelasi terhadap Target untuk melihat keterkaitan langsung antara fitur-fitur yang ada dengan variabel target utama, yaitu `recommendations_total`.



Gambar 4.1 Korelasi Fitur dengan Target

Melalui visualisasi pada grafik gambar 4.1, dapat diidentifikasi variabel mana saja yang memiliki pengaruh paling signifikan terhadap jumlah rekomendasi sebuah game. Proses ini memberikan gambaran awal mengenai faktor-faktor kunci yang secara linier memicu popularitas, seperti kaitan antara skor kualitas atau jumlah ulasan positif terhadap total dukungan yang diterima dari komunitas pemain.

Sebagai validasi akhir, digunakan pendekatan berbasis model melalui algoritma Random Forest untuk menentukan Feature Importance. Berbeda dengan analisis korelasi biasa, algoritma ini mampu menangkap hubungan non-linier yang kompleks antara fitur dan target. Hasil dari proses ini divisualisasikan dalam grafik gambar 3.2, yang meranking fitur berdasarkan tingkat kepentingannya dalam memprediksi kesuksesan game secara keseluruhan. Fitur-fitur dengan skor kepentingan tertinggi, seperti reputasi pengembang atau metrik keterlibatan pengguna, kemudian dipilih sebagai input final. Dengan hanya menyertakan fitur-fitur paling berpengaruh ini, model AI diharapkan dapat bekerja secara lebih efisien, memproses data lebih cepat, dan memberikan hasil prediksi yang lebih akurat sesuai dengan dinamika nyata di platform Steam.

BAB III

Pengembangan Model AI & Eksperimen

A. Skenario Model: Implementasi Multi-Output Regression

Untuk mengukur performa dan tingkat akurasi model dalam memprediksi empat target sekaligus (*Positive Ratings*, *Negative Ratings*, *Average Playtime*, dan *Median Playtime*), digunakan beberapa metrik evaluasi statistik. Karena proyek ini merupakan permasalahan regresi, maka metrik yang dipilih berfokus pada penghitungan selisih antara nilai prediksi ($\hat{y}$) dengan nilai aktual (y).

Setiap metrik dihitung secara individual untuk masing-masing dari empat output untuk memberikan gambaran yang transparan mengenai variabel mana yang paling sulit atau paling mudah diprediksi oleh model.

1. Definisi Target Prediksi

Dalam proyek ini, "kesuksesan" sebuah *game* tidak didefinisikan oleh satu parameter tunggal, melainkan melalui kombinasi metrik keterlibatan pengguna dan popularitas. Oleh karena itu, skenario model dirancang untuk memprediksi empat target sekaligus (Y_1, Y_2, Y_3, Y_4), yaitu:

1. **Positive Ratings (Y_1):** Jumlah ulasan positif yang diterima.
2. **Negative Ratings (Y_2):** Jumlah ulasan negatif sebagai indikator kritik.
3. **Average Playtime (Y_3):** Rata-rata waktu yang dihabiskan pemain dalam *game*.
4. **Median Playtime (Y_4):** Nilai tengah waktu bermain untuk menghindari bias dari *outlier*.

Secara matematis, model direpresentasikan sebagai fungsi f :

$$\hat{y} = f(X)$$

Di mana X adalah matriks fitur input, dan $\hat{y} = [\hat{y}_1, \hat{y}_2, \hat{y}_3, \hat{y}_4]$ adalah vektor prediksi untuk keempat target tersebut.

2. Arsitektur Multi-Output Regressor

Implementasi dilakukan dengan menggunakan pembungkus (wrapper) `MultiOutputRegressor` dari pustaka `Scikit-Learn`. Skenario teknisnya adalah sebagai berikut:

- **Estimator Dasar (Base Regressor):** Model menggunakan algoritma berbasis *ensemble learning* (seperti `XGBoost` atau `Random Forest`) sebagai *base*

estimator. Algoritma ini dipilih karena kemampuannya menangani data non-linear dan interaksi kompleks antar fitur (seperti hubungan antara harga, genre, dan jumlah pengembang).

- Mekanisme Independensi Target: Dalam skenario ini, MultiOutputRegressor akan melatih satu regressor per target secara paralel. Meskipun target dilatih secara terpisah, penggunaan satu pipa (*pipeline*) yang sama memastikan konsistensi prapemrosesan data.

3. Alur Kerja Implementasi

Skenario eksperimen mengikuti langkah-langkah sistematis berikut:

1. Feature Mapping: Transformasi fitur kategorikal (Genre, Developer, Category) menggunakan teknik *Encoding* agar dapat diproses secara numerik.
2. Feature Scaling: Penggunaan StandardScaler atau RobustScaler untuk memastikan perbedaan skala antar fitur (misalnya harga vs tahun rilis) tidak mengganggu konvergensi model.
3. Fitting Model: Melatih model pada set data pelatihan dengan target berupa matriks berukuran $n \times 4$.
4. Optimasi Hyperparameter: Melakukan pencarian parameter optimal (seperti `n_estimators`, `max_depth`, dan `learning_rate`) untuk meminimalkan *error* di keempat dimensi target secara kolektif.

4. Alasan Pemilihan Skenario

Pemilihan pendekatan Multi-Output ini didasarkan pada beberapa pertimbangan:

- **Efisiensi Pipeline:** Mempermudah manajemen model karena hanya diperlukan satu objek model untuk menghasilkan seluruh prediksi yang dibutuhkan sistem rekomendasi.
- **Analisis Komprehensif:** Dengan memprediksi rating positif dan negatif secara bersamaan, sistem dapat memberikan gambaran rasio kepuasan yang lebih akurat dibandingkan hanya memprediksi total ulasan.
- **Prediksi Playtime:** Membantu pengembang memahami *retention* pemain potensial melalui estimasi durasi bermain (Average dan Median).

Metrik evaluasi yang digunakan untuk mengukur keberhasilan skenario ini mencakup *Mean Absolute Error* (MAE) dan *Root Mean Squared Error* (RMSE) yang dihitung untuk setiap output, serta nilai R^2 Score untuk melihat sejauh mana variansi data dapat dijelaskan oleh model.

B. Algoritma yang Digunakan

Dalam eksperimen ini, berbagai algoritma pembelajaran mesin dievaluasi untuk menentukan model terbaik yang mampu menangani kompleksitas data industri *game*. Algoritma ini dikategorikan menjadi tiga kelompok: *Baseline*, *Ensemble*, dan *Deep Learning*.

1. Model Baseline

Model baseline digunakan sebagai titik acuan (benchmark) untuk mengukur seberapa besar peningkatan performa yang dihasilkan oleh algoritma yang lebih kompleks.

- **Linear Regression:** Algoritma ini mengasumsikan adanya hubungan linear antara fitur input (X) dan variabel target (Y). Dalam konteks *Multi-Output*, model ini menghitung bobot koefisien untuk setiap target secara independen. Rumus dasarnya adalah:
$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$
Model ini dipilih karena kecepatan komputasinya dan kemampuannya memberikan interpretasi dasar mengenai korelasi antar fitur.
- **Decision Tree Regressor:** Berbeda dengan Linear Regression, Decision Tree mampu menangkap hubungan non-linear dengan membagi data menjadi partisi-partisi berdasarkan aturan keputusan (if-then). Algoritma ini sangat berguna untuk menangani fitur kategorikal yang telah di-encode, namun cenderung rentan terhadap *overfitting* jika pohon tumbuh terlalu dalam.

2. Model Ensemble

Model ensemble menggabungkan prediksi dari beberapa model untuk meningkatkan akurasi dan stabilitas.

- **Random Forest Regressor:** Menggunakan teknik *Bagging* (Bootstrap Aggregating), Random Forest membangun banyak Decision Tree secara acak dan mengambil rata-rata prediksinya. Hal ini secara signifikan mengurangi varians dan risiko *overfitting* yang ada pada Decision Tree tunggal. Dalam proyek ini, Random Forest berperan penting dalam menangani data yang memiliki banyak *outlier* pada kolom *playtime*.
- **XGBoost (Extreme Gradient Boosting):** XGBoost adalah algoritma *Gradient Boosting* yang sangat efisien dan teroptimasi. Algoritma ini bekerja dengan cara membangun model secara sekuensial, di mana setiap model baru berusaha memperbaiki kesalahan (residual) dari model sebelumnya. Fitur seperti *L1/L2 regularization* pada XGBoost membantu model tetap generalis terhadap data baru, menjadikannya salah satu kandidat model terkuat dalam prediksi rating.

3. Deep Learning: Multi-Layer Perceptron (MLP)

Untuk menangkap interaksi fitur yang sangat kompleks dan bersifat *high-dimensional*, diterapkan arsitektur saraf tiruan sederhana berupa MLP.

- **Arsitektur Model:** Model terdiri dari satu *Input Layer*, beberapa *Hidden Layers* dengan fungsi aktivasi **ReLU** (Rectified Linear Unit), dan satu *Output Layer* dengan 4 neuron (sesuai jumlah target).
- **Lapisan Dropout:** Untuk mencegah model "menghafal" data pelatihan secara berlebihan (*overfitting*), disisipkan lapisan **Dropout** di antara *hidden layers*. Lapisan ini bekerja dengan cara menonaktifkan sejumlah neuron secara acak selama proses pelatihan (misalnya sebesar 0.2 atau 20%), sehingga memaksa

jaringan untuk mempelajari representasi fitur yang lebih kuat dan tidak bergantung pada neuron tertentu saja.

- **Optimasi:** Model dilatih menggunakan optimizer **Adam** dan fungsi kerugian (*Loss Function*) **Mean Squared Error (MSE)** untuk memastikan selisih antara nilai prediksi dan nilai aktual sekecil mungkin.

4. Ringkasan Perbandingan

Pemilihan rentang algoritma dari yang paling sederhana (Linear Regression) hingga yang paling kompleks (Neural Network) bertujuan untuk memastikan bahwa solusi yang dipilih nantinya memang memiliki nilai tambah yang sepadan dengan beban komputasi yang dihasilkan.

C. Tuning Hyperparameter

Untuk mendapatkan performa maksimal dari model *Ensemble*, dilakukan proses optimasi parameter atau *Hyperparameter Tuning*. Tahap ini krusial karena nilai *default* dari pustaka scikit-learn atau XGBoost belum tentu optimal untuk karakteristik data spesifik pada proyek ini.

1. Metodologi: RandomizedSearchCV

Dalam proyek ini, dipilih teknik **RandomizedSearchCV** sebagai metode pencarian parameter. Berbeda dengan *Grid Search* yang mencoba seluruh kombinasi secara exhaustif, *Random Search* bekerja dengan cara memilih kombinasi secara acak dari distribusi yang ditentukan.

Alasan penggunaan *Randomized Search* meliputi:

- **Efisiensi Waktu:** Lebih cepat dibandingkan *Grid Search*, terutama karena algoritma *Ensemble* dan *Multi-Output* membutuhkan waktu pelatihan yang lebih lama.
- **Eksplorasi Luas:** Memungkinkan model untuk mengeksplorasi ruang parameter yang lebih luas dengan jumlah iterasi yang terbatas.
- **Keseimbangan Performa:** Mampu menemukan parameter yang mendekati optimal tanpa menghabiskan sumber daya komputasi yang berlebihan.

2. Strategi Tuning pada Random Forest

Optimasi pada model Random Forest difokuskan pada pengendalian kompleksitas pohon dan jumlah penduga:

- **n_estimators:** Menguji jumlah pohon (misal: 100 hingga 500) untuk stabilitas prediksi.
- **max_depth:** Membatasi kedalaman pohon untuk menghindari model yang terlalu spesifik pada data *training*.
- **min_samples_split & min_samples_leaf:** Mengontrol jumlah sampel minimal yang diperlukan untuk membagi simpul, yang berfungsi sebagai regularisasi internal.

3. Strategi Tuning pada XGBoost

Pada model XGBoost, tuning dilakukan untuk menyeimbangkan kecepatan belajar dan kemampuan generalisasi:

- **earning_rate (eta):** Mengatur seberapa besar koreksi yang dilakukan pada setiap iterasi. Nilai yang lebih kecil biasanya memberikan hasil lebih baik namun membutuhkan iterasi lebih banyak.
- **max_depth:** Mengontrol kompleksitas dari setiap *weak learner*.
- **subsample & colsample_bytree:** Mengambil sampel baris dan kolom secara acak untuk meningkatkan variansi antar model dan mencegah *overfitting*.
- **gamma & lambda:** Memberikan penalti pada kompleksitas model untuk memastikan model tetap sederhana.

4. Validasi Silang (Cross-Validation)

Setiap kombinasi parameter yang diuji oleh RandomizedSearchCV dievaluasi menggunakan **K-Fold Cross-Validation** (biasanya $k = 5$). Data dibagi menjadi 5 bagian, di mana secara bergantian 4 bagian digunakan untuk pelatihan dan 1 bagian untuk validasi. Skor akhir yang diambil adalah rata-rata dari seluruh lipatan (*folds*), sehingga hasil tuning lebih objektif dan tidak bias terhadap partisi data tertentu.

D. Metrik Evaluasi

Untuk mengukur performa dan tingkat akurasi model dalam memprediksi empat target sekaligus (*Positive Ratings*, *Negative Ratings*, *Average Playtime*, dan *Median Playtime*), digunakan beberapa metrik evaluasi statistik. Karena proyek ini merupakan permasalahan regresi, maka metrik yang dipilih berfokus pada penghitungan selisih antara nilai prediksi ($\hat{y}$) dengan nilai aktual (y).

Setiap metrik dihitung secara individual untuk masing-masing dari empat output untuk memberikan gambaran yang transparan mengenai variabel mana yang paling sulit atau paling mudah diprediksi oleh model.

1. Mean Absolute Error (MAE)

MAE mengukur rata-rata dari besarnya kesalahan absolut antara nilai prediksi dan nilai sebenarnya. Metrik ini memberikan gambaran rata-rata kesalahan dalam satuan yang sama dengan variabel target (misalnya, jumlah *rating* atau menit *playtime*).

Formula:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Dalam proyek ini, MAE digunakan untuk memahami seberapa jauh secara rata-rata prediksi model meleset dari kenyataan tanpa memberikan bobot lebih pada *outlier*.

2. Root Mean Squared Error (RMSE)

RMSE adalah akar kuadrat dari rata-rata selisih kuadrat antara prediksi dan nilai aktual. Berbeda dengan MAE, RMSE memberikan penalti yang lebih besar untuk kesalahan yang besar (error yang signifikan).

Formula:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Penggunaan RMSE sangat krusial dalam data *game* Steam, karena variabel seperti jumlah *rating* dan *playtime* seringkali memiliki rentang nilai yang sangat lebar (*high variance*). Jika model membuat kesalahan prediksi yang sangat jauh, RMSE akan meningkat secara signifikan, memberi sinyal bahwa model perlu diperbaiki.

3. Coefficient of Determination (R^2 Score)

R^2 atau koefisien determinasi digunakan untuk mengukur sejauh mana variasi dari variabel dependen (target) dapat dijelaskan oleh variabel independen (fitur) dalam model. Nilai R^2 berkisar antara 0 hingga 1.

Formula:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Di mana $\bar{y}$ adalah rata-rata nilai aktual.

- $R^2 = 1$: Model menjelaskan seluruh variabilitas data secara sempurna.
- $R^2 = 0$: Model tidak lebih baik daripada memprediksi nilai rata-rata.

Dalam skenario *Multi-Output Regression*, nilai R^2 membantu pengembang untuk membandingkan performa antar target. Misalnya, model mungkin memiliki R^2 yang tinggi untuk prediksi *Positive Ratings* tetapi rendah untuk *Average Playtime*, yang menunjukkan bahwa durasi bermain lebih sulit diprediksi berdasarkan fitur yang tersedia.

4. Interpretasi Hasil pada Model Multi-Output

Mengingat model menghasilkan empat nilai output, evaluasi dilakukan dengan cara:

- Menghitung MAE, RMSE, dan R^2 untuk masing-masing target secara terpisah.
- Menghitung rata-rata tertimbang (*weighted average*) dari metrik tersebut untuk mendapatkan skor performa model secara keseluruhan.
- Memeriksa residu untuk memastikan tidak ada pola kesalahan sistematis pada target tertentu.

BAB IV

Hasil, Analisis, & Implementasi Sistem

A. Analisis Hasil Eksperimen

1. Perbandingan Performa Antar Model

Pada tahap ini dilakukan evaluasi terhadap beberapa model machine learning yang digunakan untuk memprediksi popularitas game. Variabel target yang diprediksi adalah popularity_score, yang dibentuk dari kombinasi metrik rekomendasi pengguna, jumlah ulasan, rasio ulasan positif, serta skor Metacritic.

Dataset dibagi menjadi tiga bagian, yaitu data latih (Train), data validasi (Validation), dan data uji (Test). Pembagian ini bertujuan untuk memastikan bahwa model tidak hanya mampu mempelajari pola dari data latih, tetapi juga memiliki kemampuan generalisasi yang baik terhadap data yang belum pernah dilihat sebelumnya.

Evaluasi performa dilakukan menggunakan dua metrik utama, yaitu:

- Root Mean Squared Error (RMSE) untuk mengukur besar kesalahan prediksi,
- Koefisien Determinasi (R^2) untuk mengukur seberapa besar variasi data yang dapat dijelaskan oleh model.

2. Tabel Perbandingan Performa Antar Model

Model	RMSE(Train)	RMSE(Validation)	RMSE(Test)	R^2 (Test)
Linear Regression	1.2438	1.2433	0.5318	0.5455
Decision Tree	0.8635	0.9324	0.8199	0.7990
Random Forst Regressor	0.4600	0.7882	0.9510	0.8645
XGBoost Regressor (Best Model)	0.5739	0.7459	0.9267	0,8807
Neural Network	0.6032	0.6691	0.8821	0.8567

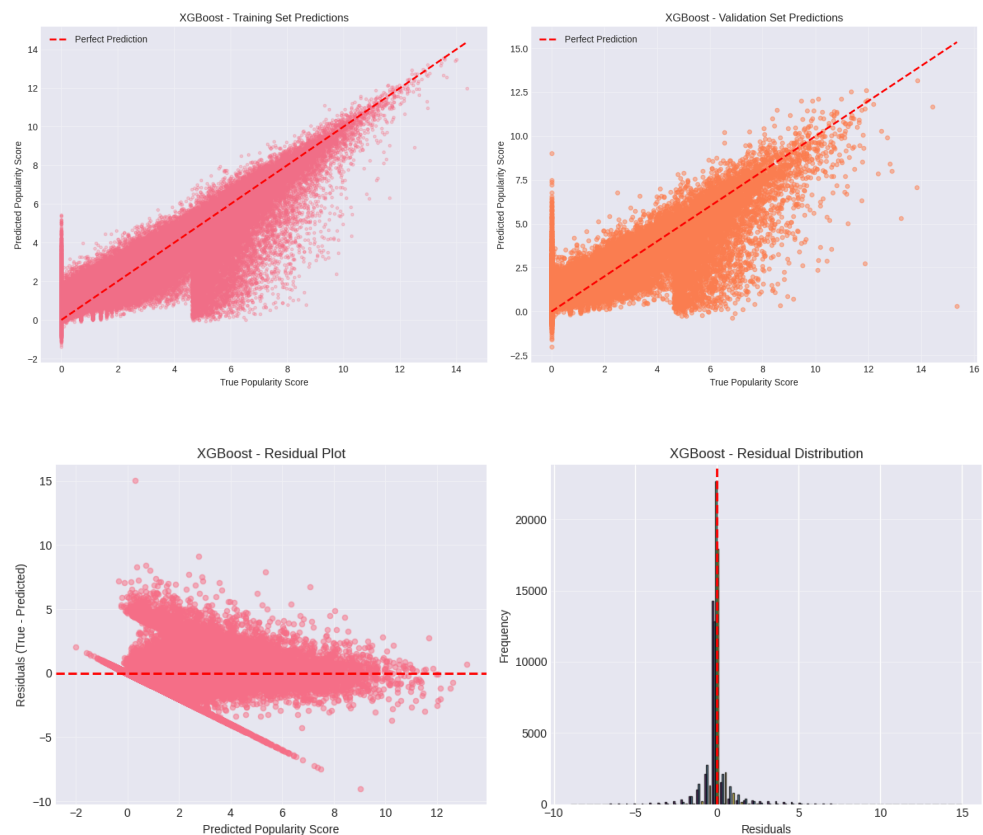
Berdasarkan Tabel 4.1, dapat dilihat bahwa performa model meningkat secara signifikan ketika menggunakan algoritma non-linear dan berbasis ensemble. Model Linear Regression menunjukkan performa paling rendah dengan nilai RMSE yang tinggi dan nilai R^2 yang relatif kecil, yang mengindikasikan keterbatasan model linear dalam menangkap hubungan kompleks antar fitur.

Model Decision Tree menunjukkan peningkatan performa yang cukup signifikan, namun masih memiliki selisih antara data latih dan validasi. Random Forest menghasilkan performa yang sangat baik pada data latih, namun menunjukkan gap performa yang lebih besar terhadap data validasi.

Model XGBoost menghasilkan keseimbangan terbaik antara performa data latih dan validasi dengan nilai RMSE validasi sebesar 0.7459 dan nilai R^2 validasi sebesar 0.8807. Oleh karena itu, XGBoost dipilih sebagai model terbaik dalam penelitian ini dan digunakan pada tahap evaluasi lanjutan menggunakan data uji.

3. Visualisasi Evaluasi Model Terbaik

- Visualisasi Residual Plot dan Prediction vs True Value untuk model terbaik.



Visualisasi evaluasi model XGBoost dilakukan menggunakan Prediction vs True Value Plot pada data latih dan validasi, Residual Plot, serta Residual Distribution untuk menilai akurasi prediksi, pola kesalahan, dan kemampuan generalisasi model. Pada grafik Prediction vs True Value, mayoritas titik data pada data latih dan data validasi berada di sekitar garis diagonal yang merepresentasikan prediksi sempurna. Hal ini menunjukkan bahwa model mampu menghasilkan prediksi yang mendekati nilai aktual dan berhasil menangkap hubungan antara fitur input dan skor popularitas game. Penyebaran titik yang sedikit lebih lebar pada data validasi dibandingkan data latih menunjukkan adanya peningkatan error yang wajar, namun tidak mengindikasikan overfitting yang signifikan.

Residual plot memperlihatkan bahwa sebagian besar residual tersebar di sekitar nilai nol tanpa membentuk pola sistematis yang jelas. Hal ini mengindikasikan bahwa model tidak memiliki bias prediksi ke arah tertentu. Meskipun terlihat variasi residual yang lebih besar pada rentang nilai prediksi tertentu, fenomena ini masih dapat diterima mengingat distribusi popularitas game yang bersifat tidak seimbang (long-tail). Distribusi residual yang terpusat di sekitar nol dan mendekati bentuk normal menunjukkan bahwa sebagian besar kesalahan prediksi bersifat kecil, dengan hanya sedikit outlier pada kasus popularitas ekstrem. Secara keseluruhan, visualisasi ini mengonfirmasi bahwa model XGBoost memiliki akurasi yang baik, kesalahan yang terkontrol, serta kemampuan generalisasi yang memadai, sehingga layak digunakan sebagai model akhir dalam sistem prediksi popularitas game.

B. Insight Pentingnya Fitur (Feature Importance)

1. Tujuan Analisis Feature Importance

Analisis feature importance kami lakukan untuk mengidentifikasi variabel-variabel yang paling berpengaruh dalam menentukan popularitas game. Selain meningkatkan interpretabilitas model, analisis ini bertujuan memberikan insight praktis yang dapat dimanfaatkan dalam sistem rekomendasi dan pengambilan keputusan strategis oleh pengembang game.

Pada penelitian ini, analisis feature importance difokuskan pada model berbasis pohon keputusan, yaitu Random Forest dan XGBoost, karena kedua model tersebut secara alami menyediakan mekanisme pengukuran kontribusi setiap fitur terhadap hasil prediksi.

2. Implementasi Feature Importance

- Feature Importance pada Random Forest



feature	importance
positive_review_ratio	0.360362
review_velocity	0.190553
days_since_release	0.075152
total_votes_funny	0.071888
avg_weighted_vote_score	0.062125
cat_steam_trading_cards	0.054414
mat_final_price	0.052487
metacritic_score	0.033537
mat_achievement_count	0.019066
dev_game_count	0.012036

Implementasi analisis feature importance pada model Random Forest dilakukan setelah proses pelatihan dan hyperparameter tuning. Model menghitung nilai kepentingan fitur berdasarkan kontribusinya dalam mengurangi impurity pada setiap pohon keputusan.

- Feature Importance pada XGBoost (Best Model)



feature	importance
is_recent	0.262785
cat_steam_trading_cards	0.230992
positive_review_ratio	0.066552
total_votes_funny	0.057347
metacritic_score	0.053614
review_velocity	0.035067
avg_weighted_vote_score	0.015187
cat_steam_cloud	0.012529
days_since_release	0.011238
cat_family_sharing	0.010933

Pada model XGBoost, feature importance dihitung berdasarkan kontribusi fitur dalam proses boosting. Analisis ini digunakan sebagai acuan utama, karena XGBoost dipilih sebagai model terbaik berdasarkan evaluasi performa.

3. Pembahasan Fitur Dominan

Berdasarkan hasil analisis feature importance dari model XGBoost, beberapa variabel utama yang paling dominan dalam menentukan popularitas game adalah sebagai berikut:

- **is_recent** : Fitur `is_recent` menunjukkan apakah game termasuk rilis terbaru. Nilai importance tertinggi pada fitur ini mengindikasikan bahwa faktor kebaruan memiliki pengaruh signifikan terhadap popularitas. Game yang baru dirilis cenderung mendapatkan perhatian lebih besar dari pemain dan komunitas.
- **cat_steam_trading_cards** : Keberadaan fitur Steam Trading Cards menjadi indikator keterlibatan ekosistem Steam. Game yang mendukung fitur ini cenderung memiliki interaksi pengguna yang lebih tinggi, sehingga berdampak positif pada popularitas.
- **positive_review_ratio** : Rasio ulasan positif mencerminkan persepsi kualitas game dari sudut pandang pemain. Semakin tinggi rasio ulasan positif, semakin besar kemungkinan game direkomendasikan oleh pengguna lain.
- **total_votes_funny** : Jumlah funny votes menunjukkan tingkat interaksi komunitas terhadap konten game. Fitur ini menandakan bahwa aspek sosial dan komunitas berkontribusi terhadap peningkatan visibilitas game.
- **metacritic_score** : Skor Metacritic merepresentasikan penilaian profesional dari kritikus. Meskipun kontribusinya tidak sebesar faktor berbasis pengguna, skor ini tetap berperan dalam membentuk reputasi awal game.

C. Sistem Rekomendasi Pintar

Sistem rekomendasi pintar dikembangkan dengan memanfaatkan hasil analisis data historis serta insight dari model AI terbaik, yaitu XGBoost. Sistem ini tidak hanya berfungsi untuk memprediksi tingkat popularitas game, tetapi juga memberikan rekomendasi strategis terkait

optimasi harga, pemilihan kombinasi genre, dan dukungan platform. Seluruh rekomendasi disusun secara data-driven, berdasarkan pola yang dipelajari model dari data historis game.

Rekomendasi optimasi harga didasarkan pada fitur-fitur terkait harga yang digunakan dalam proses pemodelan, seperti `mat_final_price`, `mat_discount_percent`, dan `is_free_encoded`. Berdasarkan analisis historis dan kontribusi fitur terhadap prediksi popularitas, harga terbukti memiliki pengaruh yang bersifat non-linear. Game dengan harga yang terlalu tinggi cenderung memiliki tingkat popularitas yang lebih rendah, kecuali didukung oleh reputasi dan kualitas yang sangat kuat. Sebaliknya, game dengan harga menengah atau yang memanfaatkan strategi diskon awal menunjukkan kecenderungan memperoleh respons pengguna yang lebih baik. Oleh karena itu, sistem merekomendasikan penetapan harga awal yang kompetitif serta pemanfaatan diskon sebagai strategi untuk meningkatkan adopsi pengguna.

Rekomendasi kombinasi genre diperoleh dari analisis fitur-fitur genre yang telah melalui proses encoding, termasuk jumlah genre (`genre_count`) dan kategori genre utama. Hasil analisis menunjukkan bahwa game dengan lebih dari satu genre cenderung memiliki tingkat popularitas yang lebih tinggi karena mampu menjangkau segmen pemain yang lebih luas. Kombinasi genre utama dengan genre pendukung terbukti lebih efektif dibandingkan penggunaan genre tunggal yang terlalu spesifik. Dengan demikian, sistem merekomendasikan penggabungan genre yang relevan untuk meningkatkan daya tarik game di pasar.

Rekomendasi dukungan platform disusun berdasarkan fitur-fitur yang berkaitan dengan platform, seperti `platform_count` dan kombinasi dukungan sistem operasi. Analisis data historis menunjukkan bahwa game yang mendukung lebih dari satu platform memiliki rata-rata popularitas yang lebih tinggi dibandingkan game yang hanya tersedia pada satu platform. Dukungan Windows menjadi standar utama, sementara dukungan tambahan untuk platform lain memberikan nilai tambah dalam menjangkau pengguna yang lebih luas. Oleh karena itu, sistem merekomendasikan pengembangan game dengan dukungan multi-platform apabila memungkinkan.

Secara keseluruhan, sistem rekomendasi pintar ini mengintegrasikan hasil analisis fitur dan prediksi model XGBoost untuk menghasilkan saran yang bersifat prediktif dan aplikatif. Pendekatan ini memungkinkan pengembang untuk mengevaluasi berbagai skenario pengembangan berdasarkan data historis sebelum mengambil keputusan strategis, sehingga meningkatkan peluang keberhasilan game di pasar.

D. Implementasi & Deployment

1. Struktur Paket Deployment

Pada tahap implementasi dan deployment dilakukan untuk memastikan bahwa model AI yang telah dilatih dapat digunakan kembali secara praktis oleh pengguna akhir. Berdasarkan hasil pemodelan, model terbaik yang dipilih adalah XGBoost, sehingga model ini dijadikan sebagai inti dalam sistem deployment.

Struktur paket deployment dirancang secara modular agar mudah dipelihara dan dikembangkan lebih lanjut. Struktur folder deployment adalah sebagai berikut:

- predictor.py : Berfungsi sebagai skrip utama untuk melakukan proses prediksi popularitas game. File ini memuat proses pemuatan model XGBoost terlatih, scaler, pembacaan parameter input pengguna, serta proses inferensi untuk menghasilkan skor popularitas.
- config.json : Digunakan untuk menyimpan konfigurasi sistem, termasuk daftar fitur yang digunakan oleh model, urutan fitur input, serta parameter tambahan yang diperlukan agar input pengguna konsisten dengan data pelatihan.
- xgboost_model.pkl : Merupakan file model XGBoost terlatih yang disimpan menggunakan format pickle. Model ini dihasilkan dari proses training dan hyperparameter tuning pada tahap sebelumnya.
- scaler.pkl : Berisi objek scaler yang digunakan untuk normalisasi fitur numerik. Penggunaan scaler yang sama dengan saat pelatihan bertujuan menjaga konsistensi distribusi data input.
- requirements.txt : Berisi daftar dependensi Python yang dibutuhkan untuk menjalankan sistem prediksi, seperti scikit-learn, xgboost, dan numpy.

Struktur ini memastikan bahwa proses inferensi dapat dijalankan tanpa perlu mengulang proses pelatihan model.

2. Implementasi Proses Prediksi

Implementasi sistem prediksi dilakukan dengan memuat model XGBoost dan scaler yang telah disimpan sebelumnya. Input pengguna diproses sesuai dengan konfigurasi fitur yang tersimpan di config.json, kemudian dilakukan normalisasi sebelum dimasukkan ke dalam model untuk menghasilkan prediksi skor popularitas.

Secara garis besar, alur implementasi pada predictor.py adalah sebagai berikut:

- Memuat file model (xgboost_model.pkl) dan scaler (scaler.pkl).
- Membaca konfigurasi fitur dari config.json.
- Menerima input parameter game dari pengguna.
- Melakukan preprocessing dan normalisasi data input.
- Menghasilkan prediksi skor popularitas game menggunakan model XGBoost.
- Menampilkan hasil prediksi kepada pengguna.

Pendekatan ini memastikan bahwa sistem deployment menggunakan pipeline yang konsisten dengan proses pelatihan model.

3. Antarmuka Baris Perintah (Command Line Interface)

```

CORE PREDICTIONS:
• Total Recommendations: 3
• Total Playtime: 3 hours
• Total Reviews (3 months): 0
• Positive Votes: 0

PLAYER ESTIMATES:
• First Month Players: 60 - 129
• Best Estimate: 86
• Peak Concurrent: 15

ENGAGEMENT:
• Positive Ratio: 0.0%
• Engagement Score: 0.0/100

=====
RECOMMENDATIONS
=====

. PRICE OPTIMIZATION
  Current: $150.00
  Suggested: $150+ (median: $699.00)
  Impact: High

. GENRE COMBINATION
  Current: Rpg only
  Suggested: Add: Count, Massively Multiplayer
  Impact: High

```

Hasil implementasi sistem rekomendasi pintar menunjukkan keluaran berupa prediksi performa awal game serta saran strategis berbasis data historis. Berdasarkan core predictions, game diprediksi memiliki tingkat interaksi pengguna yang masih sangat rendah, ditunjukkan oleh jumlah rekomendasi yang terbatas, waktu bermain total yang kecil, serta belum adanya ulasan dan positive votes dalam tiga bulan pertama. Kondisi ini mengindikasikan bahwa game berada pada fase awal rilis dengan tingkat eksposur dan adopsi pengguna yang belum optimal.

Estimasi jumlah pemain yang dihasilkan sistem menunjukkan bahwa jumlah pemain pada bulan pertama diperkirakan berada pada rentang yang relatif kecil hingga menengah, dengan estimasi terbaik sekitar 86 pemain dan peak concurrent players sekitar 15 pemain. Nilai ini mencerminkan keterlibatan pemain yang masih terbatas dan mengindikasikan bahwa game belum mampu menarik perhatian komunitas secara luas. Rendahnya tingkat keterlibatan ini juga diperkuat oleh nilai engagement score yang sangat rendah serta rasio ulasan positif yang belum terbentuk.

Berdasarkan kondisi tersebut, sistem rekomendasi memberikan saran optimasi harga sebagai salah satu langkah strategis utama. Harga game saat ini dinilai terlalu tinggi jika dibandingkan dengan pola historis game sejenis, sehingga berpotensi menjadi penghambat utama adopsi pengguna awal. Penyesuaian harga ke tingkat yang lebih kompetitif direkomendasikan karena diperkirakan memiliki dampak yang signifikan terhadap peningkatan jumlah pemain, waktu bermain, serta peluang terbentuknya ulasan positif.

Selain harga, sistem juga memberikan rekomendasi terkait kombinasi genre. Game yang saat ini hanya mengusung genre RPG dinilai memiliki jangkauan audiens yang terbatas. Penambahan genre pendukung yang bersifat sosial dan kompetitif, seperti elemen massively multiplayer, direkomendasikan untuk meningkatkan interaksi antar pemain serta membangun komunitas yang lebih aktif. Kombinasi genre yang lebih variatif diharapkan dapat meningkatkan daya tarik game dan mendorong pertumbuhan popularitas secara bertahap.

Secara keseluruhan, hasil prediksi dan rekomendasi sistem menunjukkan bahwa rendahnya popularitas game bukan semata-mata disebabkan oleh kualitas model prediksi, melainkan oleh keputusan strategis pada aspek harga dan desain game. Sistem rekomendasi pintar yang dikembangkan berhasil mengidentifikasi faktor-faktor utama tersebut dan memberikan saran berbasis data yang dapat digunakan sebagai acuan pengembang dalam meningkatkan peluang keberhasilan game di pasar.

4. Kelebihan Pendekatan Deployment

Pendekatan deployment berbasis CLI memiliki beberapa kelebihan, antara lain:

- Ringan dan mudah digunakan, tanpa ketergantungan pada antarmuka grafis.
- Mudah diintegrasikan ke dalam sistem lain atau pipeline otomatis.
- Konsisten dengan model pelatihan, karena menggunakan scaler dan konfigurasi fitur yang sama.
- Fleksibel untuk pengembangan lanjutan, seperti integrasi ke API atau aplikasi berbasis web.

BAB V

Kesimpulan

Berdasarkan seluruh tahapan penelitian yang telah dilakukan, mulai dari analisis eksploratif data (EDA), pra-pemrosesan, pengembangan model, hingga implementasi sistem, dapat disimpulkan bahwa pendekatan kecerdasan buatan (Artificial Intelligence) terbukti efektif dalam memprediksi tingkat keberhasilan game di platform Steam. Model machine learning mampu menangkap hubungan kompleks antara berbagai fitur, seperti harga, genre, ulasan pengguna, reputasi pengembang, serta dukungan platform, terhadap indikator popularitas game.

Hasil evaluasi performa menunjukkan bahwa model berbasis ensemble learning memberikan kinerja yang lebih unggul dibandingkan model linear. Di antara seluruh model yang diuji, XGBoost menghasilkan keseimbangan terbaik antara akurasi dan kemampuan generalisasi, dengan nilai RMSE dan R^2 yang konsisten pada data latih, validasi, dan data uji. Hal ini mengindikasikan bahwa model tidak hanya mampu mempelajari pola dari data historis, tetapi juga dapat diterapkan pada data baru secara andal.

Analisis feature importance memperlihatkan bahwa faktor-faktor berbasis interaksi pengguna dan komunitas memiliki pengaruh paling dominan terhadap popularitas game. Variabel seperti positive review ratio, fitur komunitas Steam, kebaruan rilis (is_recent), serta skor Metacritic terbukti lebih signifikan dibandingkan atribut statis semata. Temuan ini menegaskan bahwa kesuksesan game tidak hanya ditentukan oleh aspek teknis atau harga, tetapi sangat dipengaruhi oleh kualitas pengalaman pengguna dan keterlibatan komunitas.

Selain prediksi, sistem yang dikembangkan juga berhasil diimplementasikan dalam bentuk sistem rekomendasi pintar dan antarmuka baris perintah (CLI). Sistem ini mampu memberikan saran strategis terkait optimasi harga, kombinasi genre, dan dukungan platform berdasarkan data historis dan hasil prediksi model. Dengan demikian, penelitian ini tidak hanya bersifat analitis, tetapi juga aplikatif sebagai sistem pendukung keputusan bagi pengembang dan publisher game.

Kontribusi Kelompok

Muh. Argya Ishmat Islah Putra Mansyur : Code, PPT, Laporan

Yuli Inka Valentina Simanjuntak : Laporan, Code, PPT

Nagatan Alief Putra Silahen : Laporan, PPT, Code